Create and Publish an OData V4 Service in SAP

A complete beginner-friendly workbook: install Java, Eclipse and ADT; connect to SAP BTP ABAP Environment; create RAP objects; publish and test the API.

Audience: Developers and integration engineers who know APIs but are new to ABAP development.

Target: A read-only OData V4 Web API exposing product data through CDS and RAP.

Important: Trial availability and menu names can change. Follow current SAP requirements for your environment. Publishing a binding does not automatically make it publicly accessible.

Learning path

1. Architecture and prerequisites
2. SAP BTP trial and URLs
3. Java, Eclipse and ADT installation
4. Connect Eclipse
5. Create package and table
6. Create CDS view entity
7. Create service definition
8. Create and publish OData V4 Web API binding
9. Test and troubleshoot
10. External access and security

1. Understand what you are building

A consumer sends an HTTP GET request and receives product records as JSON. RAP provides the standard service infrastructure; you do not hand-code an HTTP endpoint.

Object	Purpose	Example
Database table	Stores product records.	`ZCUST_DEMO_T`
CDS view entity	Defines the fields and key exposed.	`ZI_CUST_DEMO`
Service definition	Declares the public entity set.	`ZUI_CUST_DEMO`
Service binding	Chooses protocol/type and publishes the endpoint.	`ZUI_CUST_DEMO_O4`

Flow: HTTP client → OData V4 binding → service definition → CDS view entity → database table.

> **Read-only vs writable:** This guide exposes data for reading. Create/update/delete operations require RAP behavior definitions and implementation, authorizations, and additional testing.

2. Key terms

Term	Plain-English meaning
SAP BTP	SAP Business Technology Platform, where the cloud ABAP environment runs.
ABAP	SAP's programming language.
ABAP Cloud	Modern ABAP development with rules about released APIs.
Eclipse	Desktop development environment.
ADT	ABAP Development Tools plug-in for Eclipse.
CDS	Core Data Services; a structured data model over tables or CDS entities.
RAP	ABAP RESTful Application Programming Model, SAP's framework for OData services.
OData	A standard for exposing data over HTTP with metadata and query options.
Service definition	Declares which CDS entities are exposed.
Service binding	Binds the definition to OData V4 Web API or another supported protocol/type.
$metadata	The XML schema describing entity types, properties, data types and keys.
Activate	Checks and activates a development object.
Publish	Enables an active binding for use in the current ABAP system.

3. Prerequisites

• An SAP BTP account with an ABAP Environment trial or licensed ABAP environment.

• Developer permissions to create objects and publish service bindings.

• Internet access and a desktop supported by Eclipse/ADT.

• A JDK and Eclipse version compatible with the ADT release. Check SAP's current requirements first.

> Trial plans, regions, quotas and lifecycle behavior can change. If ABAP Environment is not offered in your account, check current SAP trial availability rather than repeatedly changing Eclipse settings.

4. Prepare SAP BTP ABAP Environment

4.1 Sign in and locate the instance

• Sign in to SAP BTP Cockpit and open the relevant trial subaccount.

• Locate the ABAP Environment instance or follow SAP's current trial setup tutorial.

• If an instance already exists and works, reuse it; do not delete it just to follow this guide.

• If creating a new instance, select a plan actually offered to your account and complete the setup wizard.

4.2 Identify the correct URL

Use the ABAP service instance URL when creating the Eclipse project. Service keys may contain more than one URL. The top-level `url` points to the ABAP system; `uaa.url` points to the authentication server and is not the ABAP project URL.

Never paste service-key client secrets or bearer tokens into screenshots, public chat, source control or documentation. Rotate/recreate credentials if exposed.

5. Install Java, Eclipse and ADT

5.1 Install a compatible JDK

• Read the current ADT installation requirements and install a supported JDK.

• Open a terminal/command prompt and run:

```
java -version
```

The command should print the installed Java version. If Java is not found, install the JDK or correct PATH/JAVA_HOME before proceeding.

5.2 Install Eclipse

• Download Eclipse from the official Eclipse Foundation website.

• Install or extract it, launch Eclipse, and choose a workspace folder. A workspace stores Eclipse settings and project metadata.

5.3 Install ADT

• In Eclipse, open Help → Install New Software.

• Use SAP's official ADT update-site page: https://tools.hana.ondemand.com/. Select the update site matching your Eclipse/ADT release.

• Select the ABAP Development Tools features, continue through the wizard, accept the license, install, and restart Eclipse.

• After restart, confirm that the ABAP Cloud Project wizard is available.

If installation fails: Check that Eclipse and ADT releases match. Some Eclipse packages lack dependencies such as EMF. Install the requested dependencies from the matching Eclipse release repository or use a compatible Eclipse package; do not mix unrelated Eclipse releases.

6. Connect Eclipse to SAP

• Choose File → New → Other → ABAP Cloud Project (menu wording may vary), then Next.

• Enter the ABAP service instance URL.

• Choose Open Logon Page in Browser and complete SAP sign-in.

• If the wrong SAP account is signed in to your default browser, use Copy Logon URL and open it in a private window.

• Return to Eclipse and finish. The system should appear in Project Explorer.

If the wizard is missing, verify ADT installation. If the URL is rejected, confirm that it is the ABAP URL rather than the UAA URL.

7. Create a development package

• In Project Explorer, right-click the appropriate package node and choose New → ABAP Package.

• Create a package such as `ZCUSTOMER_APP`, or reuse your existing package.

• In a trial, you may need a provided superpackage such as `ZLOCAL`; use the values offered by the system.

• Choose a Development package for development objects. A Structure package is only a container.

• Finish the wizard and activate/save if prompted.

Z and Y are common customer namespaces. Object names must be unique and comply with your system's naming rules.

8. Create the database table

This is a practice table. In client projects, data often already exists, so you may instead build a CDS view over an existing table or released CDS view.

• Right-click the development package → New → Other ABAP Repository Object.

• Search for Database Table and create `ZCUST_DEMO_T`.

• Replace the editor contents with this definition and activate it:

```
define table zcust_demo_t {
  key client     : abap.clnt not null;
  key product_id : abap.char(10) not null;
  name           : abap.char(40);
```

```
  category       : abap.char(20);
  price          : abap.dec(15,2);
  in_stock       : abap.char(1);
}
```

Field	Purpose
CLIENT	SAP client field required by this example table.
PRODUCT_ID	Product identifier; part of the primary key.
NAME	Product name.
CATEGORY	Product category.
PRICE	Decimal price with two decimal places.
IN_STOCK	Practice flag: X means yes; blank means no.

If activation fails, start with the first syntax error. Check spelling, object type, package assignment, field types and punctuation.

9. Add sample data (optional)

The API can be published with an empty table, but the collection will then be empty. Add a few practice rows using a data-entry method supported by your environment. Options vary by release and permissions.

> Do not run DELETE or bulk test-data scripts against client or production tables. Avoid duplicate primary keys, and let the ABAP environment handle client context as required.

Product ID	Name	Category	Price	In stock
M001	CNC Machine	Machine	12500.00	X
M002	Lathe Machine	Machine	8400.00	X
M003	Hydraulic Press	Machine	9100.00	

10. Create the CDS view entity

• Right-click the package → New → Other ABAP Repository Object.

• Search for Data Definition and create `ZI_CUST_DEMO`.

• Enter the source below and activate it:

```
@EndUserText.label: 'Customer Demo Products'
@AccessControl.authorizationCheck: #NOT_REQUIRED
define view entity ZI_CUST_DEMO
  as select from zcust_demo_t
{
  key product_id as ProductId,
      name       as Name,
      category   as Category,
      price      as Price,
      in_stock   as InStock
}
```

Code	Meaning
`define view entity`	Creates a modern CDS view entity.
`as select from zcust_demo_t`	Reads data from the practice table.
`key product_id`	Defines the entity key used by the API.
`as ProductId`	Makes ProductId the public property name.
`#NOT_REQUIRED`	Simplifies this demo; not a production security recommendation.

The client field is not exposed as a business property. ABAP's client-dependent object semantics govern access to client data.

> **Production:** design row-level access control with DCL and the appropriate authorization setting. Do not use this demo's relaxed authorization configuration for sensitive business data.

11. Create the service definition

• Right-click the package → New → Other ABAP Repository Object → Service Definition.

• Create `ZUI_CUST_DEMO`.

• Enter the following and activate:

```
@EndUserText.label: 'Customer Demo OData Service'
define service ZUI_CUST_DEMO {
  expose ZI_CUST_DEMO as Products;
}
```

The alias `Products` becomes the public entity-set name. Treat it and the field names as an API contract; changing them later can break consumers.

12. Create and publish the OData V4 Web API binding

12.1 Create the binding

• Right-click the package → New → Other ABAP Repository Object → Business Services → Service Binding.

• Set the name to `ZUI_CUST_DEMO_O4` and add a description.

• Choose **OData V4 - Web API** as Binding Type. This is intended for API consumers; do not choose OData V4 - UI for a plain machine-to-machine API.

• Select `ZUI_CUST_DEMO` as the service definition and finish the wizard.

• Activate the binding.

12.2 Publish

• Open the binding editor and choose Publish.

• Wait for the status to update, then copy the Service URL shown by the editor.

> Use the actual URL displayed by your system; do not guess it from the object name. The binding must be active before publishing. Publishing enables the service in the current ABAP system but does not automatically configure external authentication.

If OData V4 - Web API is not offered, check system release/capabilities and developer permissions. Do not silently choose V2 if the requirement is specifically V4.

13. Test the service

13.1 Test in ADT

• Open the active service binding.

• Use Test if available. Depending on the ADT/system release, an API explorer or Swagger-like page may open.

• Inspect the Service URL and the entity set, then execute a GET request.

13.2 Try OData V4 requests

Start with the exact Service URL from the binding editor. Append the entity set or query suffix appropriate to that root URL:

Test	Example suffix	Expected result
Metadata	`$metadata`	EDMX/XML schema with properties and keys.
List	`Products`	JSON collection; empty if there are no rows.
Limit	`Products?$top=2`	At most two records.
Filter	`Products?$filter=Category eq 'Machine'`	Only matching products.
Projection	`Products?$select=ProductId,Name`	Only requested properties.
Sort	`Products?$orderby=Price desc`	Price descending.
Count	`Products?$count=true`	Collection response includes count when supported.

OData V4 does not use V2-only `$inlinecount=allpages`. V4 commonly uses `$count=true`.

> HTTP 200 with JSON is a good sign. HTTP 200 with HTML that redirects to a login page is not an OData JSON result. A 401/403 should be investigated as authentication/authorization, separately from CDS syntax.

14. Read the responses

Result	Interpretation
200 + JSON collection	The service returned data.
200 + empty collection	Endpoint may be correct, but there may be no data or no matching rows.
401 Unauthorized	Authentication was not accepted or not supplied.
403 Forbidden	The user may be authenticated but lacks permission or access is denied.
404 Not Found	Check URL, entity-set alias, and binding publication status.
HTML login page	Often an authentication redirect, not the OData payload.
Activation error	Resolve syntax or dependency errors before testing.

15. Troubleshooting checklist

Problem	What to check
ABAP Cloud Project wizard missing	ADT installation; restart Eclipse.
Connection fails	Correct ABAP URL, correct SAP account, instance status.
Cannot create objects	Developer role, permissions, package type and package assignment.
Structure package error	Create objects in a Development package.
CDS activation error	Table name, field names, aliases, syntax and active dependencies.
Service definition error	CDS view active and exposed name correct.
V4 binding type missing	Release/capabilities and permissions.
Publish unavailable	Activate binding first; verify authorization.
No rows returned	Insert practice data; check correct system/client and filter.
401/403	Diagnose authentication, authorization and communication setup separately.
HTML instead of JSON	Inspect redirects and use the correct machine-to-machine auth flow.
Local ADT proxy error	A local proxy failure does not prove the backend service is broken; test via the binding's supported test feature.

16. External access is a separate step

There are two milestones:

- **Milestone A:** The binding is published and the service works in ADT's supported test interface.
- **Milestone B:** An external application can call the service using a supported authentication method and appropriate permissions.

> For some ABAP Cloud Web API scenarios, external access requires a communication scenario and communication arrangement, plus a communication user/system. Follow the SAP instructions for the exact release and API. The Publish button alone is not a complete external integration setup.

Getting an OAuth token does not prove it is valid for the OData resource: audience, scopes, user context and authorization must also match. Do not assume an interactive browser-login token flow works for machine-to-machine integration.

> Never put secrets or bearer tokens in workflow prompts, browser code, repositories or screenshots. Use your integration platform's secret storage and rotate exposed credentials.

17. Read-only versus write support

Requirement	Typical additional work
Read/list	CDS view entity + service definition + OData V4 binding.
Filter/sort/page	Use supported standard OData query options.
Create/update/delete	RAP behavior definition and implementation, authorizations, and write/CSRF testing where applicable.
External API access	Supported authentication plus communication configuration and permissions.

18. Final checklist

- ■ JDK compatible with the chosen ADT release installed.
- ■ Eclipse launches and a workspace is selected.
- ■ ADT installed from SAP's official update site.
- ■ ABAP Cloud Project connects to the correct system.
- ■ Development package created.
- ■ Database table activates; practice rows added if desired.
- ■ CDS view entity activates and has a key.
- ■ Service definition activates and exposes Products.
- ■ Binding type is OData V4 - Web API.
- ■ Binding is active and published.
- ■ Service URL copied from the binding editor.
- ■ $metadata and collection GET requests tested.
- ■ External authentication/communication configured separately if needed.
- ■ No secrets or tokens exposed.

19. Official learning links

Create an ABAP Cloud Project (SAP Developer Center)
https://developers.sap.com/tutorials/abap-environment-create-abap-cloud-project

Connect to the ABAP System (SAP Help)
https://help.sap.com/docs/btp/sap-business-technology-platform/connect-to-abap-system

Creating Service Binding (ADT User Guide)
https://help.sap.com/docs/abap-cloud/abap-development-tools-user-guide/4bad0d7280b742a68dd2d42b1441f09a.html

Service Binding overview (SAP Help)
https://help.sap.com/docs/abap-cloud/abap-rap/service-binding

Creating an OData V4 Service (SAP Learning)
https://learning.sap.com/courses/getting-started-with-creating-an-sap-fiori-elements-app-based-on-an-odata-v4-rap-service/creating-an-odata-v4-service

Building an ABAP RESTful Application (SAP Learning)
https://learning.sap.com/courses/learn-the-basics-of-abap-programming-on-sap-btp/building-an-abap-restful-application

End state: You have created a read-only OData V4 Web API, published it in the ABAP system, and tested the schema and collection. External consumption is a separate authentication and communication task.